

Contents

1	DD09: 監査 action コード (運営者システム)	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	1
1.4.1	3.1 命名規約	1
1.4.2	3.2 全 action コード一覧 (約 60 件)	2
1.4.3	3.3 action コード使用ルール	4
1.4.4	3.4 3 区分保持の物理対応 (action_code × retention_class)	4
1.4.5	3.5 retention_class 別の物理削除契機	5
1.4.6	3.6 action コード追加プロセス	5
1.4.7	3.7 監査記録の必須項目	5
1.5	4. 関連設計	6
1.6	5. テスト観点	6
1.6.1	5.1 ユニットテスト	6
1.6.2	5.2 結合テスト (Miniflare)	6
1.6.3	5.3 CI 整合性チェック	6
1.6.4	5.4 月次運用テスト	6
1.7	6. 未確定事項・確認事項	6

1 DD09: 監査 action コード (運営者システム)

1.1 0. 文書情報

項目	内容
文書名	DD09: 監査 action コード (運営者システム)
詳細設計 ID	DD09
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	TH-6, NFR-602, D-08
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	TH-6	監査 action コード一覧の確定
テーブル	audit_logs.action	全 action コードの列挙
関連実装	admin/src/shared/action-codes.ts	TypeScript 列挙体

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 15 全文	監査 action コード一覧 (★TH-6)	命名規約 + 全 action コード約 60 件 + 使用ルール
付録 F	3 区分保持の物理対応 (action_code × retention_class)	1y / 5y / 7y マップ

1.4 3. 詳細設計本文

<resource>.<verb> 命名規約で全 action コードを確定する。

1.4.1 3.1 命名規約

項目	規約
形式	<resource>.<verb> または <resource>.<sub-resource>.<verb>
文字	英小文字 + ハイフン + ドット + アンダースコア
動詞	過去形は使わない (create/update/delete/approve/reject/execute 等)

1.4.2 3.2 全 action コード一覧 (約 60 件)

action コード	retention	4-eyes 対象	payload スキーマ参照
owner.*			
owner.suspend	5y	MVP Log Only / Beta Hard Gate	{contractOwnerId, reason, suspendedAt}
owner.restore	5y	MVP Log Only / Beta Hard Gate	{contractOwnerId, restorationId, reason, rollback}
owner.physical_delete	5y	MVP Hard Gate	{contractOwnerId, slug, reason, ticketId}
owner.restore_data	5y	MVP Log Only / Beta Hard Gate	{contractOwnerId, resourceType, resourceId, reason, rollback}
owner.update	5y	-	{contractOwnerId, before, after}
operator.*			
operator.invite	5y	-	{invitedId, invitedBy, email}
operator.accept	5y	-	{operatorId}
operator.disable	5y	-	{operatorId, reason}
operator.session.revoke	5y	-	{operatorId, sessionId}
operator.login.attempt	1y	-	{email, success, ipMasked}
operator.login.success	5y	-	{operatorId, sessionId}
operator.login.failed	5y	-	{email, failedCount}
operator.lockout	5y	-	{operatorId, lockedUntil}
operator.password.reset.request	5y	-	{email}
operator.password.reset	5y	-	{operatorId, approvalId}
operator.subrole.grant	5y	-	{operatorId, subrole}
operator.subrole.revoke	5y	-	{operatorId, subrole}
operator_approval.*			
operator_approval.request	5y	-	{approvalId, actionCode, payloadHash}
operator_approval.start	1y	review	{approvalId, reviewerId}
operator_approval.approve	5y	-	{approvalId, approvedBy}
operator_approval.reject	5y	-	{approvalId, rejectedBy, comment}
operator_approval.withdraw	1y	-	{approvalId, withdrawnBy, comment?}
operator_approval.execute	5y	-	{approvalId, executedBy, result}
operator_approval.expire	5y	-	{approvalId}
operator_mfa.*			
mfa.setup	5y	-	{operatorId}
mfa.verify	5y	-	{operatorId, success}
mfa.recovery_code.use	5y	-	{operatorId, codeIdMasked}
reauth	5y	-	{operatorId, reauthId}
operator_ip.*			
operator_ip.grant	5y	-	{operatorId, cidr}
operator_ip.revoke	5y	-	{operatorId, cidr}
ai_parameter.*			
ai_parameter.update	5y	MVP Hard Gate	{scope, scopelId, before, after, version}
ai_model.switch	5y	-	{from, to, rolloutPercentage}
rate_limit.* / budget_limit.*			
rate_limit.override	5y	MVP Log Only / Beta Hard Gate	{contractOwnerId, before, after, overrideId}

action コード	retention	4-eyes 対象	payload スキーマ参照
budget_limit.override	5y	MVP Log Only / Beta Hard Gate	{contractOwnerUserId, before, after, overrideId}
suppress_list.restore	5y	MVP Log Only	{contractOwnerUserId, email}
widget.force_stop	5y	MVP Log Only / Beta Hard Gate	{contractOwnerUserId, reason}
announcement.*			
announcement.create	5y	-	{announcementId, kind, severity, scope}
announcement.preview	1y	-	{announcementId}
announcement.schedule	5y	-	{announcementId, scheduledAt}
announcement.cancel	5y	-	{announcementId, cancelledBy}
announcement.test_send	5y	-	{announcementId, sentTo}
announcement.send	5y	-	{announcementId, recipients}
announcement.dispatch	5y, dlq	-	{announcementId, attemptCount, failedReason}(自動 BO 3 回失敗で永久失敗)
announcement.correct	5y, on.issue	-	{newAnnouncementId, correctionOf}
webhook.*			
webhook.receive	7y	-	{eventId, eventType, payloadHash}
webhook.replay	5y	-	{eventId, replayId, attemptedBy}
webhook.signature.invalid	5y	-	{ipMasked, attemptedAt}
webhook.payload_diff.detect	5y	-	{diffId, eventId, originalHash, newHash}
webhook.payload_diff.review	5y	-	{diffId, reviewedBy}
webhook.payload_diff.reprocess	5y	-	{diffId, replayId}
webhook.payload_diff.dismiss	5y	-	{diffId, reason}
billing.*			
billing.invoice.issued	7y	-	{invoiceId, contractOwnerUserId, amount, billingYearMonth}
billing.invoice.finalized	7y	-	{invoiceId}
billing.credit_note.issue	7y	-	{creditNoteId, invoiceId, amount, reason}
billing.cron.run	7y	-	{month, success, failed}
pricing.update	7y	MVP Log Only / Beta Hard Gate	{pricingVersion, before, after, effectiveFrom}
stripe.*			
stripe.event.processed	7y	-	{eventId, eventType}
stripe.subscription.resume	5y	-	{subscriptionId, contractOwnerUserId}
pii.*			
pii_fp_report.create	1y	-	{reportId, detectionLayer}
pii_fp_report.transition	5y	-	{reportId, from, to}
pii_rule.update	5y	-	{revisionId, rolloutPercentage}
audit.*			
audit.export	5y	-	{exportId, format, rows}
audit.search	1y	-	{operatorId, filter, hitCount, ticketId}(APPI 30 条: 監査ログ閲覧自体の記録)
audit.chain.verify	5y	-	{verifyDate, runId, mismatchCount}
audit.chain.verify.fail	5y	-	{runId, rowId, expected, actual}
master_key.*			
master_key.rotate	5y	MVP Hard Gate	{rotationId, oldKeyId, newKeyId}
master_key.emergency_bypass	5y	-	{bypassId, reason, witnesses, ticketId}(RB-014 MVP の紙ベース回復コード使用記録)
retention.*			
retention.purge.run	5y	-	{runId, retentionClass, deletedCount}
retention.archive.run	5y	-	{archiveYear}
prod.*			
prod.direct_change	5y	-	{operatorId, query, ticketId}

action コード	retention	4-eyes 対象	payload スキーマ参照
mail.*			
mail.suppress.add	5y	-	{email, reason}
mail.suppress.remove	5y	-	{email, approvalId}
mail.footer.update	5y	-	{before, after, operatorId}(特定電子メール法フッタ設定変更)
feature.*			
feature.hard_gate.toggle	5y	MVP Log Only / Beta Hard Gate	{actionCode, before, after, reason}(ハードゲート KV フラグ切替)
monitoring.*			
monitoring.threshold.update	5y	-	{kpId, before, after, reason}(監視 KPI しきい値変更)

1.4.3 3.3 action コード使用ルール

1. 新規 action コード追加時は **本表に追記** + `src/shared/action-codes.ts` 定数を更新する。
2. CI で本表と実装の整合性を検証 (grep ベース + TypeScript 型)。
3. 既存 action コードの retention_class 変更は RB-018 に従う。
4. 4-eyes 対象列の変更 (ハードゲート ☒ Log Only) は KV **feature:hard-gate:<action>** の操作で行い、本表の備考を更新する。

1.4.4 3.4 3 区分保持の物理対応 (action_code × retention_class)

§ 3.2 + [DD03_ 監査ハッシュチェーン.md](#) § 3.7 と整合した完全列挙:

1.4.4.1 3.4.1 1 年保持 (NFR-602(a) 業務監査)

action_code	主管
faq.create / faq.update / faq.publish / faq.unpublish / faq.delete	メイン
chat.reply / chat.close / chat.reopen	メイン
account.login / account.logout	メイン + 本書 (本書側は operator.login.*)
inbox.read	メイン + 本書
announcement.preview	本書
announcement.test_send	本書
pii_fp_report.create	本書

1.4.4.2 3.4.2 5 年保持 (NFR-602(b) 運営者高権限)

action_code	4-eyes
owner.suspend / owner.restore / owner.physical_delete / owner.update	☒ (physical_delete のみハードゲート)
ai_parameter.update / ai_model.switch	☒ ハードゲート
rate_limit.override / budget_limit.override / suppress_list.restore / widget.force_stop	☒ (Beta から)
announcement.create / announcement.schedule / announcement.cancel / announcement.send / announcement.correction.issue	-
operator.* 全般 (invite / accept / disable / session.revoke / login.success / login.failed / logout / password.reset.request / password.reset / subrole.grant / subrole.revoke)	-
operator_approval.* 全般 (request / start_review / approve / reject / withdraw / execute / expire)	-
mfa.setup / mfa.verify / mfa.recovery_code.used / reauth	-
operator_ip.grant / operator_ip.revoke	-
webhook.replay / webhook.signature.invalid / webhook.payload_diff.*	-
pii_fp_report.transition / pii_rule.update	-
audit.export / audit.chain.verify / audit.chain.verify.fail	-
master_key.rotate	☒ ハードゲート
retention.purge.run / retention.archive.run	-

action_code	4-eyes
prod.direct_change	-
mail.suppress.add / mail.suppress.remove	-

1.4.4.3 3.4.3 7 年保持 (NFR-602(c) 課金・取引)

action_code
billing.invoice.issued / billing.invoice.finalized
billing.credit_note.issued
billing.cron.run
stripe.event.processed / stripe.subscription.resume
webhook.receive

1.4.5 3.5 retention_class 別の物理削除契機

retention_class	DELETE 契機	担当 Worker
general(1y)	occurred_at < today - 365	RetentionPurgeWorker(日次 03:00 JST)
operator_high_priv(5y)	occurred_at < today - 1825	同上
billing(7y)	occurred_at < today - 2555	同上

物理削除前に R2 アーカイブ (`audit-archive/<retention_class>/<year>/<month>.tar.gz`、年次 12/31) へ書出。詳細は [DD08_Cron 実装.md](#) § 3.2.5 / § 3.2.6 を参照。

1.4.6 3.6 action コード追加プロセス

1. 本表に新規 action コード追加 + `retention_class` + 4-eyes 対象有無を決定
2. `admin/src/shared/action-codes.ts` の TypeScript 列挙体を更新
3. `CI(03_script/check-action-codes.sh`、TODO: 整備中) で本表と実装の差分検出
4. 関連 DD(認証 / 4-eyes / 監査 / 機能領域) に追記
5. `feature.hard_gate.toggle` で運用ハードゲート切替時は KV `feature:hard-gate:<action>` の状態と本表の備考を整合

1.4.7 3.7 監査記録の必須項目

すべての action 記録時に次のフィールドを必ず設定:

カラム	内容
actor_id	操作者 ID(運営者 ID または system)
action	本表の action コード
target_type / target_id	操作対象のリソース型 + ID
payload	JSON、本表「payload スキーマ参照」列の通り
payload_hash	sha256(canonical_json(payload))
retention_class	本表第 2 列 (general / operator_high_priv / billing)
prev_hash	チェーン直前レコードの record_hash
record_hash	HMAC-SHA256 計算結果
occurred_at	UTC ISO 8601
trace_id	リクエストトレース ID(構造化ログと一致)
ticket_id	クリティカル操作のチケット ID(任意)
approval_id	4-eyes 経由実行時の operator_approvals.id(任意)

完全な DDL は [基本設計 / テーブル設計](#) を正本とする。

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
テーブル設計 (正本)	../02_ 基本設計/03_ テーブル設計.md
セキュリティ設計 (正本)	../02_ 基本設計/09_ セキュリティ設計.md
関連 DD	DD03_ 監査ハッシュチェーン.md / DD02_4-eyes 承認フロー.md / DD08_Cron 実装.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

1.6 5. テスト観点

1.6.1 5.1 ユニットテスト

- 本表と `src/shared/action-codes.ts` の TypeScript 列挙体一致 (snapshot test)
- 各 action コードの payload スキーマバリデーション
- `retention_class` 値の正当性 (`general` / `operator_high_priv` / `billing` のみ)

1.6.2 5.2 結合テスト (Miniflare)

- 各 action コード発火 → `audit_logs` 1 行 INSERT + `payload_hash` 確認
- `retention_class` 別 cutoff 計算 (`general` = 365 / `operator_high_priv` = 1825 / `billing` = 2555)
- 4-eyes 対象 action (ハードゲート 3 操作) の `approval_id` 必須化

1.6.3 5.3 CI 整合性チェック

- 本表に存在しない action コード使用で CI fail
- 実装に存在しない action コードが本表にある場合 CI warning
- `retention_class` 変更時は RB-018 手順書のリンクが必要

1.6.4 5.4 月次運用テスト

- ランダム抽出 10 件で `record_hash` 再計算 → 一致確認
- `retention` 物理削除前の最終件数 + R2 アーカイブとの突合

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済